

Java Mastery: Exceptions & Assertions

Session 1 Study Guide, Practical Lab & Assignment

SYLLABUS OVERVIEW

- What is an Exception?
- try-catch Hierarchy & Handling
- Clean-up with finally
- Multiple Catch Blocks
- Multi-Catch Syntax
- throw vs throws
- Custom Exceptions
- Try-with-resources & Assertions

1 What is an Exception?

An exception is an abnormal condition or error event that occurs while a program is running, disrupting the normal instruction flow.

```
int a = 10;
int b = 0;
int result = a / b;
```

This statement triggers:

ArithmeticException: / by zero

Normally, the runtime abruptly terminates execution. Java provides robust structured exception handling mechanisms to catch and handle these errors gracefully.

2 try-catch

Risky code that could trigger an exception is enclosed in a try block, while recovery logic is handled in the catch block.

```
try {
    // risky code
}
catch (ExceptionType e) {
    // handling code
}
```

Example:

```
public class Main {
    public static void main(String[] args) {
        try {
            int result = 10 / 0;
            System.out.println(result);
        }
        catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero");
        }
        System.out.println("Program continues...");
    }
}
```

```
Cannot divide by zero
Program continues...
```

Core Principle: Place only the potential failure operations inside `try`. The `catch` block should implement safe degradation or logging.

3 The finally Block

The `finally` block always executes—whether an exception was thrown, handled, or absent. It is primarily used for releasing external resources.

```
try {
    int result = 10 / 2;
    System.out.println(result);
}
catch (ArithmeticException e) {
    System.out.println("Error");
}
finally {
    System.out.println("Finally block executed");
}
```

```
5
Finally block executed
```

Even if an uncaught or caught exception occurs:

```
try {
    int result = 10 / 0;
}
catch (ArithmeticException e) {
    System.out.println("Error occurred");
}
finally {
    System.out.println("Finally executed");
}
```

```
Error occurred
Finally executed
```

4 Multiple Catch Blocks

When code can raise distinct exceptions, multiple sequential catch blocks handle each specific issue.

```
try {
    int[] numbers = {10, 20, 30};
    System.out.println(numbers[5]);
}
catch (ArithmeticException e) {
    System.out.println("Arithmetic problem");
}
catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Invalid array index");
}
```

```
Invalid array index
```

Subclass-First Rule: Specific (subclass) exceptions must appear before generic (superclass) exceptions like `Exception`. Placing `catch (Exception e)` first causes an unreachable code compiler error.

5 Multi-Catch (Multiple Exceptions in Single Catch)

Introduced in Java 7, multi-catch lets you combine multiple disjoint exceptions using the pipe (`|`) operator.

```
try {
    // code
}
catch (ArithmeticException | ArrayIndexOutOfBoundsException e) {
    System.out.println("Something went wrong");
}
```

This avoids repetitive boilerplate when handling multiple errors with identical logic.

6 throw

The `throw` keyword explicitly triggers an exception object instance.

```
public class Main {
    public static void main(String[] args) {
        int age = 15;
        if (age < 18) {
            throw new IllegalArgumentException("You are not eligible");
        }
        System.out.println("Eligible");
    }
}
```

Memory Hook: throw = "I am throwing an exception right here, right now."

7 throws

The throws keyword is part of a method signature. It advertises to calling methods that checked exceptions might be propagated out of this method.

```
public static void readFile() throws IOException {
    // file operation
}

public static void main(String[] args) throws IOException {
    readFile();
}
```

Keyword	Role & Definition	Placement
throw	Actually triggers/throws an exception instance immediately.	Inside method body
throws	Declares that a method signature may propagate exceptions.	Method signature header

8 Custom Exceptions

Custom exceptions represent domain-specific domain business errors by extending Exception (checked) or RuntimeException (unchecked).

```
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}

public class Main {
    static void checkAge(int age) throws InvalidAgeException {
        if (age < 18) {
            throw new InvalidAgeException("Age must be 18 or above");
        }
        System.out.println("Eligible");
    }

    public static void main(String[] args) {
        try {
            checkAge(15);
        }
        catch (InvalidAgeException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Age must be 18 or above

9 Try-with-resources

Introduced in Java 7, Try-with-resources automatically closes any resource implementing `java.lang.AutoCloseable` upon block exit.

```
// Traditional approach: requires verbose finally block with close()
try (FileReader reader = new FileReader("data.txt")) {
    int data = reader.read();
    System.out.println((char) data);
}
catch (IOException e) {
    System.out.println("File error");
}
```

The `FileReader` is reliably closed even if exceptions occur during read operations.

10 Assertions

Assertions allow developers to verify internal logic assumptions during development and testing.

```
// Basic syntax
assert condition;

// With descriptive message
int age = 20;
assert age >= 18 : "Age must be at least 18";
System.out.println("Age is valid");
```

Execution Flag: Assertions are disabled by default in the JVM. Enable them using the `-ea` flag:

```
java -ea Main
```

Session 1 Practical Practice Questions

Question 1 — Division

Create a program that takes two numbers, divides the first by the second, handles division by zero (`ArithmeticException`), and prints *"Division successful"* when no error occurs.

Question 2 — Array Access

Given `int[] numbers = {10, 20, 30, 40, 50};`, prompt the user for an array index and print the value. Handle `ArrayIndexOutOfBoundsException` safely.

Question 3 — Multiple Exceptions

Build a program that accepts user inputs capable of triggering `ArithmeticException`, `ArrayIndexOutOfBoundsException`, and `NumberFormatException`. Handle each gracefully with distinct catch blocks.

Question 4 — throw Keyword

Write a method `checkBalance(double balance)`. If `balance` is negative (< 0), throw an `IllegalArgumentException("Invalid balance")`.

Question 5 — Custom Exception ★

Define a custom exception `InvalidMarksException`. If marks entered are < 0 or > 100 , throw the exception with a helpful warning message (e.g., *"Marks must be between 0 and 100"*).



Session 1 Assignment: Student Result Management System

Assignment Specifications

Construct a robust, console-driven Java application that prompts for:

- Student Name
- Marks for **Java**, **SQL**, and **Python**

Mandatory Requirements:

1. **Marks Validation:** Each subject mark must be within $0 - 100$. Otherwise, raise `InvalidMarksException`.
2. **Calculations:** Compute Total Marks, Average, and Percentage.
3. **Grading Scale:**
 - 90–100 → Grade A
 - 80–89 → Grade B
 - 70–79 → Grade C
 - 60–69 → Grade D
 - Below 60 → Grade F
4. **Robust Input Validation:** Prevent crash on non-numeric inputs (e.g. entering "abc"). Catch `InputMismatchException` or `NumberFormatException`.
5. **Syntax Checklist:** Must incorporate `try`, `catch`, `finally`, `throw`, `throws`, a **Custom Exception**, and a **multi-catch** block.



Session 1 Mini Test (Self-Check)

1. What is the fundamental difference between `throw` and `throws`?
2. What is the exact purpose of the `finally` block?
3. What is the terminal output of this snippet?

```
try {  
    System.out.println(10 / 0);  
}  
catch (ArithmeticException e) {  
    System.out.println("Error");  
}  
finally {  
    System.out.println("Done");  
}
```

4. Why and when do enterprise applications create custom exceptions?
5. What interface must an object implement to work with Try-with-resources?
6. What is multi-catch syntax and what benefit does it provide?
7. What is an assertion and how is it enabled at runtime?

Next Milestone: Complete the 5 practice questions and the Student Result Management System assignment before proceeding to **Session 2: java.lang Package** (deep dive into Strings, Regular Expressions, Pattern/Matcher, and quantifiers).